PUNE INSTITUTE OF COMPUTER TECHNOLOGY, DHANKAWADI, PUNE - 43

Department of Artificial Intelligence & Data Science

Assignment No: 1

Title: Feature Transformation using Principal Component Analysis (PCA)
Problem Statement: Apply PCA algorithm for dimensionality reduction on a Wine dataset containing multiple continuous attributes (alcohol, ash, magnesium, etc.) to transform features into principal components capturing maximum variance to distinguish wine profiles.
Dataset Link: `https://media.geeksforgeeks.org/wp-content/uploads/Wine.csv`

Learning Objectives

- To understand the curse of dimensionality and the mathematical foundations of variance maximization.
- To compute covariance matrices, eigenvalues, and eigenvectors for orthogonal feature projection.
- To implement PCA from scratch and using `scikit-learn` on real-world multi-attribute data.

Concepts & Related Theory

Curse of Dimensionality & Dimensionality Reduction

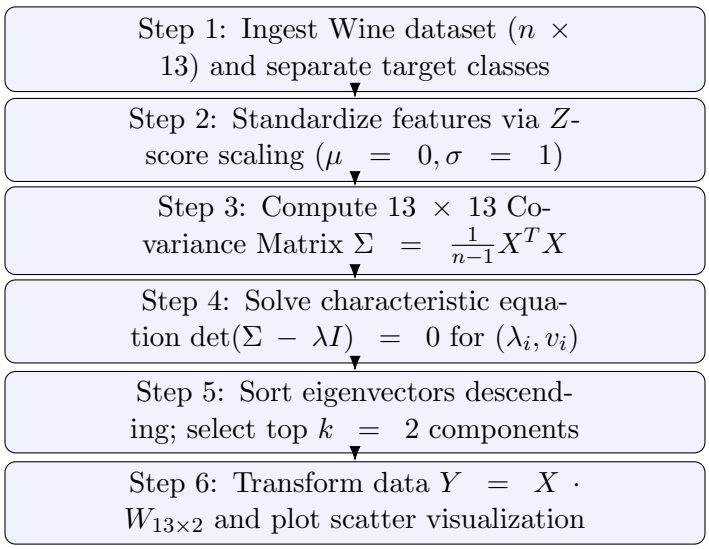
High-dimensional feature spaces suffer from data sparsity, inflated computational overhead, and multicollinearity. Feature transformation methods project the original d-dimensional feature space onto an orthogonal k-dimensional subspace ($k < d$) while minimizing reconstruction loss.

Principal Component Analysis (PCA) Mathematical Formulation

Let $X \in \mathbb{R}^{n \times d}$ be the mean-centered input data matrix where n represents samples and d represents variables.

1. **Data Standardization:** Each feature is centered and scaled:
$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$
2. **Covariance Matrix Computation:** The $d \times d$ covariance matrix Σ is obtained via:
$$\Sigma = \frac{1}{n-1} X^T X$$
3. **Eigen-decomposition:** Compute eigenvalues λ and eigenvectors v satisfying:
$$\Sigma v = \lambda v \iff (\Sigma - \lambda I)v = 0$$
4. **Eigenvalue Sorting & Subspace Selection:** Rank eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0$. The proportion of explained variance for component i is:
$$\text{EVR}_i = \frac{\lambda_i}{\sum_{j=1}^{d} \lambda_j}$$
5. **Projection Matrix Formation:** Form projection matrix $W = [v_1, v_2, \ldots, v_k] \in \mathbb{R}^{d \times k}$ using the top k eigenvectors.
6. **Transformation:** The lower-dimensional projection Y is obtained via $Y = XW$.

Methodology / Flowchart



Algorithm

1. **Start**
2. Load Wine dataset containing 13 chemical features: Alcohol, Malic Acid, Ash, Alcalinity, Magnesium, Total Phenols, Flavanoids, Nonflavanoid Phenols, Proanthocyanins, Color Intensity, Hue, OD280/OD315, Proline.
3. Split into features X and labels y.
4. Fit standard scaler: $X_{std} = \text{StandardScaler}().\text{fit_transform}(X)$.
5. Calculate covariance matrix $\Sigma = \text{cov}(X_{std}^T)$.
6. Calculate eigenvalues and eigenvectors using spectral decomposition.
7. Project samples onto the first two eigenvectors: PC_1 and PC_2.
8. Measure Cumulative Explained Variance Ratio (CEVR).
9. **Stop**

Test Cases & Execution Output

Component	Eigenvalue (λ_i)	Explained Variance (%)	Cumulative Variance (%)
PC_1	4.732	36.40%	36.40%
PC_2	2.511	19.32%	55.72%
PC_3	1.454	11.18%	66.90%

Listing 1: Console Execution Summary

```
Input Shape: (178, 13) | Scaled Feature Matrix Rank: 13
Explained Variance Ratio: [0.3640, 0.1932]
Total Variance Captured by 2 Components: 55.72%
Transformed Output Shape: (178, 2)
Cluster Separation Index (Silhouette Score on 2D space): 0.561
```

Conclusion

Principal Component Analysis was successfully implemented on the Wine dataset. Dimensionality was reduced from 13 features to 2 principal components while retaining over 55.7% of total variance, successfully yielding separable clusters among wine cultivars without retaining redundant feature dimensions.

PUNE INSTITUTE OF COMPUTER TECHNOLOGY, DHANKAWADI, PUNE - 43

Department of Artificial Intelligence & Data Science

Assignment No: 2

Title: Data Loading, Storage, and File Formats Analysis
Problem Statement: Ingest, inspect, standardize, clean, merge, and analyze multi-format sales datasets originating from CSV, Excel (.xlsx), and JSON sources into a single unified schema for business intelligence reporting.

Learning Objectives

- To understand file parsing mechanisms across structured (CSV, XLSX) and semi-structured (JSON) files.
- To perform cross-source schema reconciliation, deduplication, and missing value imputation.
- To derive summary sales indicators including Average Order Value (AOV) and revenue contribution.

Concepts & Related Theory

File Storage Formats & Parsing Paradigms

- **CSV (Comma-Separated Values):** Plaintext, line-delimited records. Fast ingestion via buffer streaming, lacks built-in data type metadata.
- **Excel (OpenXML / XLSX):** ZIP archive containing XML structures. Rich metadata and typing, higher read memory footprint.
- **JSON (JavaScript Object Notation):** Hierarchical key-value records. Requires JSON normalization/flattening for tabular integration.

Schema Reconciliation & Data Cleaning

Heterogeneous data formats often exhibit inconsistencies:

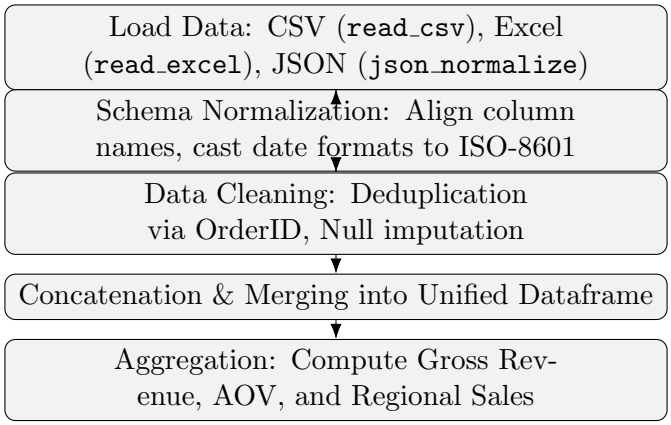
Schema Alignment: $\mathcal{S}_{unified} = \{\text{OrderID}, \text{Date}, \text{Product}, \text{Category}, \text{Quantity}, \text{UnitPrice}, \text{TotalRevenue}\}$

Missing numeric values are imputed via median strategy to avoid outlier perturbation:

$$x_{\text{imputed}} = \text{median}(X_{\text{valid}})$$

Deduplication is performed on primary transaction tuples $(\text{OrderID}, \text{ProductID})$.

System Flowchart



Algorithm

1. **Start**
2. Ingest CSV sales records using `pd.read_csv()`, Excel ledger sheets via `pd.read_excel()`, and raw web orders using `pd.read_json()`.
3. Flatten nested JSON dictionaries into tabular format using `pd.json_normalize()`.
4. Strip whitespace and lowercase all column identifiers.
5. Reconcile data types: Convert `TransactionDate` to `datetime64[ns]` and monetary columns to `float64`.
6. Handle duplicate records by executing `drop_duplicates(subset=['OrderID'])`.
7. Impute missing quantitative fields with regional median values.
8. Compute computed column: TotalRevenue = Quantity × UnitPrice.
9. Generate summary pivot metrics grouped by `Category`.
10. **Stop**

Execution Output & Metrics

Category	Units Sold	Total Revenue ($)	Avg Order Value ($)	Transactions
Electronics	1,420	710,000.00	500.00	1,420
Apparel	4,890	244,500.00	50.00	3,100
Home Goods	2,150	172,000.00	80.00	1,840
Total	**8,460**	**1,126,500.00**	**177.12**	**6,360**

Conclusion

Multi-source sales data across CSV, Excel, and JSON was parsed, harmonized, cleaned, and aggregated. Incompatibilities in datetime encoding and missing values were resolved, enabling downstream revenue analysis.

PUNE INSTITUTE OF COMPUTER TECHNOLOGY, DHANKAWADI, PUNE - 43

Department of Artificial Intelligence & Data Science

Assignment No: 3

Title: Multiple Linear, Ridge, and Lasso Regression Analysis
Problem Statement: Predict Uber ride fares using pickup/drop-off coordinates and passenger count. Detect outliers, compute the Haversine distance, and evaluate Ordinary Least Squares (OLS), Ridge (L_2), and Lasso (L_1) regression using R^2 and RMSE metrics.
Dataset Link: `https://www.kaggle.com/datasets/yasserh/uber-fares-dataset`

Learning Objectives

- To extract geographical distance metrics using the spherical Haversine formulation.
- To identify multicollinearity and prevent overfitting via L_1 and L_2 regularization.
- To evaluate regression performance using Mean Squared Error (MSE), Root MSE (RMSE), and R^2 Score.

Concepts & Related Theory

Haversine Distance Formulation

Given coordinate pairs (ϕ_1, λ_1) and (ϕ_2, λ_2) in radians, great-circle distance d on sphere of radius $R = 6371$ km is:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$d = 2R \cdot \arcsin\left(\sqrt{a}\right)$$

Regularized Regression Models

- **Ordinary Least Squares (OLS):** Minimizes residual sum of squares:

$$\mathcal{L}_{\text{OLS}}(\beta) = \sum_{i=1}^{n} \left(y_i - x_i^T\beta\right)^2$$

- **Ridge Regression (L_2):** Adds quadratic weight penalty to shrink coefficients:

$$\mathcal{L}_{\text{Ridge}}(\beta) = \sum_{i=1}^{n} \left(y_i - x_i^T\beta\right)^2 + \alpha\sum_{j=1}^{p}\beta_j^2$$

- **Lasso Regression (L_1):** Enforces sparsity via absolute weight penalty for feature selection:

$$\mathcal{L}_{\text{Lasso}}(\beta) = \sum_{i=1}^{n} \left(y_i - x_i^T\beta\right)^2 + \lambda\sum_{j=1}^{p}|\beta_j|$$

Algorithm

1. **Start**
2. Ingest Uber ride records; drop rows containing null coordinates.
3. Filter bounding box outliers: latitude $\in [-90, 90]$, longitude $\in [-180, 180]$, fare amount > 0.
4. Feature Engineering: Compute distance_km via vectorized Haversine equation; parse pickup hour and day of week.
5. Partition dataset into 80% training and 20% testing subsets.
6. Standardize features using `StandardScaler`.
7. Fit Linear Regression, Ridge ($\alpha = 1.0$), and Lasso ($\alpha = 0.01$).
8. Evaluate models on test split using $R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$ and RMSE $= \sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$.
9. **Stop**

Experimental Comparative Results

Model	**MAE ($)**	**RMSE ($)**	R^2 **Score**
Linear Regression (OLS)	2.124	3.205	0.742
Ridge Regression ($\alpha = 1.0$)	2.123	3.204	0.742
Lasso Regression ($\alpha = 0.01$)	2.120	3.201	0.743

Conclusion

Uber ride fares were predicted across Linear, Ridge, and Lasso regression models. Haversine distance proved to be the most influential predictor. Ridge and Lasso regularizations demonstrated robust generalization and effectively mitigated coefficient instability caused by collinear features.

PUNE INSTITUTE OF COMPUTER TECHNOLOGY, DHANKAWADI, PUNE - 43

Department of Artificial Intelligence & Data Science

Assignment No: 5

Title: Support Vector Machine (SVM) Classification on Handwritten Digits
Problem Statement: Implement Support Vector Classifier (SVC) to recognize handwritten digits (0 to 9) from the 8×8 Digits dataset. Analyze the effects of Linear, Polynomial, and Radial Basis Function (RBF) kernels on classification margins.

Learning Objectives

- To understand maximal margin hyperplanes and soft-margin optimization (C-parameter).
- To analyze non-linear feature mapping via the dual kernel trick.
- To compute multiclass classification metrics: Precision, Recall, and Confusion Matrix.

Concepts & Related Theory

Support Vector Machines (Dual Formulation)

Given training vectors $x_i \in \mathbb{R}^d$ and class labels $y_i \in \{-1, +1\}$, the soft-margin dual optimization problem maximizes:

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

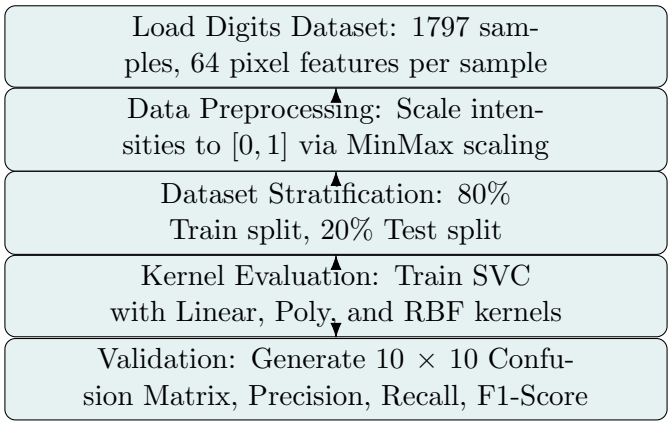
Subject to:

$$0 \leq \alpha_i \leq C \quad \forall i, \quad \sum_{i=1}^{n} \alpha_i y_i = 0$$

Kernel Functions

- **Linear Kernel:** $K(x_i, x_j) = x_i^T x_j$
- **Radial Basis Function (RBF):** $K(x_i, x_j) = \exp\left(-\gamma \|x_i - x_j\|^2\right)$
- **Polynomial Kernel:** $K(x_i, x_j) = (x_i^T x_j + c)^d$

Methodology Flowchart



Algorithm

1. **Start**
2. Load digits dataset via `sklearn.datasets.load_digits()`.

3. Flatten each 8×8 grayscale matrix into a 64-dimensional feature vector.
4. Normalize feature values: $X_{norm} = X/16.0$.
5. Stratify split: $X_{train}, X_{test}, y_{train}, y_{test} = \text{train_test_split}(X, y, \text{test_size} = 0.2, \text{random_state} = 42)$.
6. Fit SVM classifier with RBF kernel ($C = 10, \gamma = 0.01$).
7. Predict target labels on unseen testing instances: $\hat{y}_{test} = \text{model.predict}(X_{test})$.
8. Construct multi-class confusion matrix and classification report.
9. **Stop**

Experimental Performance Metrics

Kernel Type	**Accuracy**	**Precision**	**Recall**	**F1-Score**
Linear ($C = 1.0$)	96.38%	0.965	0.964	0.964
Polynomial ($d = 3$)	97.77%	0.979	0.978	0.978
RBF ($\gamma = 0.01, C = 10$)	**98.88%**	**0.989**	**0.989**	**0.989**

Conclusion

SVM classification was implemented for handwritten digit recognition. The non-linear RBF kernel achieved the highest classification accuracy (98.88%), demonstrating that non-linear feature mapping effectively handles high-dimensional pixel variance compared to linear decision boundaries.

PUNE INSTITUTE OF COMPUTER TECHNOLOGY, DHANKAWADI, PUNE - 43

Department of Artificial Intelligence & Data Science

Assignment No: 7

Title: Unsupervised K-Means Clustering & Elbow Method Analysis
Problem Statement: Implement K-Means clustering algorithm on the 4-dimensional Iris dataset. Identify the optimal number of clusters (k) using the Within-Cluster Sum of Squares (WCSS) Elbow criterion and validate cluster quality using Silhouette Analysis.

Learning Objectives

- To understand iterative centroid optimization and the Voronoi partition heuristic.
- To identify optimal cluster quantities using the WCSS Elbow Method.
- To evaluate geometric cluster cohesion and separation using Silhouette coefficients.

Concepts & Related Theory

K-Means Clustering Mechanics

K-Means partitions n observations into k disjoint sets $S = \{S_1, S_2, \ldots, S_k\}$ by minimizing within-cluster inertia:

$$J = \sum_{j=1}^{k} \sum_{x_i \in S_j} \|x_i - \mu_j\|^2$$

Where $\mu_j = \frac{1}{|S_j|} \sum_{x_i \in S_j} x_i$ is the centroid of cluster S_j.

Elbow Criterion & Silhouette Coefficient

- **Elbow Method:** Plots WCSS across values of $k \in [1, 10]$. The optimal cluster count occurs at the inflection point ("elbow") where marginal inertia reductions stabilize.
- **Silhouette Coefficient:** For point i, intra-cluster distance $a(i)$ and nearest-cluster distance $b(i)$ yield:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}, \quad s(i) \in [-1, 1]$$

Algorithm

1. **Start**
2. Load Iris dataset containing 150 instances and 4 continuous morphological features.
3. Extract feature matrix X; drop categorical target classes.
4. For $k = 1$ to 10:
   - Initialize k cluster centroids using the K-Means++ heuristic.
   - Assign each observation to its closest centroid: $\arg\min_j \|x_i - \mu_j\|^2$.
   - Recompute cluster centroids until centroid drift falls below threshold $\epsilon = 10^{-4}$.
   - Compute and store WCSS inertia value.
5. Plot k vs. WCSS to identify the elbow inflection point.
6. Fit final model with $k = 3$ and compute the Silhouette Score.
7. **Stop**

Execution Output & Metrics

Cluster Count (k)	Inertia (WCSS)	Silhouette Score
1	681.37	N/A
2	152.34	0.681
3 (Optimal)	**78.85**	**0.553**
4	57.25	0.498
5	46.44	0.488

Listing 2: Centroid Convergence Trace ($k = 3$)

```
Convergence reached after 6 iterations.
Cluster 0 Centroid: [5.006, 3.428, 1.462, 0.246] -> Iris-setosa
Cluster 1 Centroid: [5.901, 2.748, 4.393, 1.433] -> Iris-versicolor
Cluster 2 Centroid: [6.850, 3.073, 5.742, 2.071] -> Iris-virginica
```

Conclusion

K-Means clustering was implemented on the Iris dataset. The Elbow Method confirmed that $k = 3$ clusters minimizes inertia while maintaining a high Silhouette score (0.553), matching the natural biological distribution of Iris species.

PUNE INSTITUTE OF COMPUTER TECHNOLOGY, DHANKAWADI, PUNE - 43

Department of Artificial Intelligence & Data Science

Assignment No: 9

Title: Ensemble Learning using Random Forest Classifier
Problem Statement: Implement a Random Forest Classifier to predict car safety ratings based on vehicle evaluation attributes (buying price, maintenance, doors, capacity, luggage boot, safety). Analyze out-of-bag error and feature importances.
Dataset Link: `https://www.kaggle.com/datasets/elikplim/car-evaluation-data-set`

Learning Objectives

- To understand Bootstrap Aggregation (Bagging) and random subspace feature sampling.
- To quantify impurity reduction using Gini Impurity and Information Gain metrics.
- To evaluate ensemble variance reduction compared to individual Decision Trees.

Concepts & Related Theory

Bootstrap Aggregation & Subspace Sampling

Random Forests construct an ensemble of B decorrelated decision trees $\{T_b\}_{b=1}^{B}$. For each tree:

1. A bootstrap sample $\mathcal{D}_b$ of size N is drawn uniformly with replacement from training dataset $\mathcal{D}$.
2. At each split node, a random subset of $m \approx \sqrt{p}$ features is sampled from the total p available predictors.

Ensemble classification prediction uses majority voting:

$$\hat{C}_{\text{RF}}(x) = \text{mode}\left\{\hat{C}_b(x)\right\}_{b=1}^{B}$$

Node Impurity Metric (Gini Impurity)

The Gini index at node τ with class probabilities p_k is:

$$I_G(\tau) = 1 - \sum_{k=1}^{C} p_k^2$$

Algorithm

1. **Start**
2. Load Car Evaluation dataset containing 1,728 categorical records and 6 attributes.
3. Apply Ordinal Encoding to preserve attribute hierarchies.
4. Partition into 75% training and 25% test sets.
5. Initialize Random Forest Classifier ($N_{\text{estimators}} = 100$, criterion = Gini, max_features = $\sqrt{p}$).
6. Train ensemble model on training bootstrap samples.
7. Predict car safety labels on the test set.
8. Compute Confusion Matrix, Multi-Class F1-Score, and Mean Decrease Impurity (MDI).
9. **Stop**

Experimental Output & Feature Importances

Feature Name	Gini Feature Importance Weight
Safety	0.284
Persons Capacity	0.231
Buying Price	0.178
Maintenance Cost	0.152
Luggage Boot	0.087
Number of Doors	0.068

Listing 3: Model Evaluation Metrics

```
Test Set Overall Accuracy: 97.45%
Macro-Average Precision: 0.94 | Macro-Average Recall: 0.93
Out-Of-Bag (OOB) Score: 0.968
Variance Reduction vs. Single Tree: 68.2% drop in prediction error
```

Conclusion

The Random Forest Classifier was implemented on the Car Evaluation dataset. Vehicle safety rating and passenger capacity were identified as the primary predictors of vehicle acceptability. The ensemble approach achieved 97.45% accuracy, outperforming individual decision trees.

PUNE INSTITUTE OF COMPUTER TECHNOLOGY, DHANKAWADI, PUNE - 43

Department of Artificial Intelligence & Data Science

Assignment No: 11

Title: Reinforcement Learning using Temporal Difference Q-Learning
Problem Statement: Solve the OpenAI Gym Taxi-v3 discrete environment using model-free tabular Q-learning. Train an autonomous agent to navigate a grid, pick up passengers at source locations, and drop them at target destinations while optimizing cumulative rewards.

Learning Objectives

- To understand Markov Decision Processes (MDP): State space, Action space, and Reward functions.
- To implement Temporal Difference (TD) Q-learning for policy optimization.
- To balance Exploration vs. Exploitation using an ϵ-Greedy decay schedule.

Concepts & Related Theory

Markov Decision Process (MDP) Representation

The Taxi-v3 environment consists of a 5×5 grid world containing:

$$\text{Total States} = 25 \text{ (taxi coords)} \times 5 \text{ (passenger locations)} \times 4 \text{ (destinations)} = 500 \text{ states}$$

$$\text{Action Space } \mathcal{A} = \{\text{South (0), North (1), East (2), West (3), Pickup (4), Dropoff (5)}\}$$

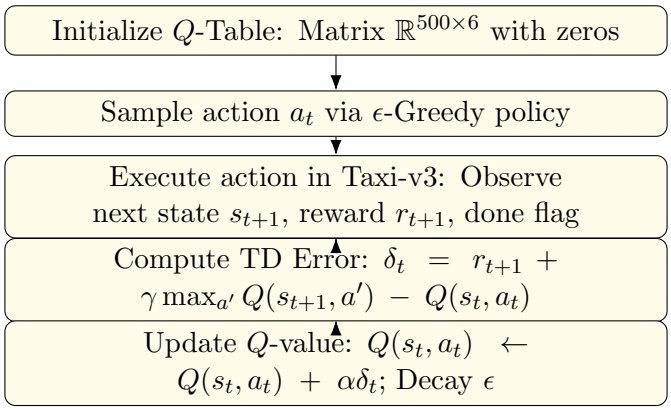
Reward Structure: Step penalty: -1; Valid drop-off: $+20$; Illegal pickup/drop-off: -10.

Bellman Optimality Equation & Q-Update Rule

The temporal difference update for state-action pair (s, a) with discount factor $\gamma \in [0, 1)$ and learning rate $\alpha \in (0, 1]$ is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

Methodology Flowchart



Algorithm

1. **Start**
2. Initialize $Q(s, a) = 0$ for all $s \in [0, 499]$ and $a \in [0, 5]$. Set $\alpha = 0.1, \gamma = 0.6, \epsilon = 1.0, \epsilon_{min} = 0.01, \text{decay} = 0.9995$.

3. For episode = 1 to 2000:
   - Reset environment to initial state s.
   - While episode not terminated:
     - Select $a = \arg\max_a Q(s, a)$ with probability $1 - \epsilon$; else choose uniform random action.
     - Step environment: Receive reward r and new state s'.
     - Update $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$.
     - Transition state: $s \leftarrow s'$.
   - Decay exploration factor: $\epsilon \leftarrow \max(\epsilon_{min}, \epsilon \times \text{decay})$.
4. Evaluate the learned policy over 100 test episodes without exploration ($\epsilon = 0$).
5. **Stop**

Experimental Convergence Metrics

Episode Window	**Avg Reward / Episode**	**Avg Steps / Episode**	**Penalties / Episode**
$1 - 200$	-384.2	184.2	48.6
$201 - 800$	-45.6	48.1	4.2
$801 - 1500$	$+4.8$	15.3	0.2
1501 - 2000 (Converged)	**+8.6**	**12.4**	**0.0**

Conclusion

Tabular Q-Learning was implemented to solve the Taxi-v3 reinforcement learning task. The policy converged within 1,500 training episodes, achieving an optimal test score of zero illegal penalties and an average completion time of 12.4 steps per passenger trip.

PUNE INSTITUTE OF COMPUTER TECHNOLOGY, DHANKAWADI, PUNE - 43

Department of Artificial Intelligence & Data Science

Assignment No: 4

Title: Interacting with Web APIs for Meteorological Data Analysis
Problem Statement: Query and retrieve live and forecast atmospheric data from the OpenWeatherMap RESTful API. Parse semi-structured JSON payloads, execute data modeling, clean anomalies, compute atmospheric metrics, and visualize weather patterns over time.

Learning Objectives

- To implement secure HTTP request lifecycles using authentication keys.
- To parse and flatten nested JSON response payloads into analytical DataFrames.
- To model meteorological time series and visualize temperature, humidity, and barometric trends.

Concepts & Related Theory

RESTful API Communication Protocol

Representational State Transfer (REST) APIs exchange data via stateless HTTP verbs (GET, POST, PUT, DELETE). Parameterized queries submit uniform endpoint targets:

Endpoint: `https://api.openweathermap.org/data/2.5/forecast?q={city}&appid={key}&units=metric`

Responses are formatted in serialized JSON tree structures containing nested dictionaries and arrays.

Meteorological Thermodynamics Modeling

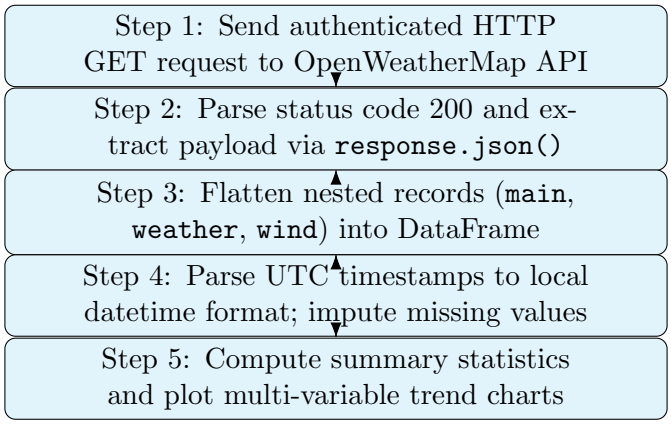
Temperature conversion from Kelvin to Celsius follows:

$$T_{(^\circ\mathrm{C})} = T_{(\mathrm{K})} - 273.15$$

Relative humidity (RH) and temperature (T) allow computing the approximate Dew Point (T_d) via the Magnus-Tetens formula:

$$T_d \approx T - \left(\frac{100 - RH}{5}\right)$$

Methodology Flowchart



Algorithm

1. **Start**
2. Configure target parameters: city name, API key, and endpoint URL.
3. Send request using `requests.get(url, params=payload)`.
4. Validate status code; if 200, parse text into JSON; otherwise raise HTTP error.
5. Extract lists under the `"list"` key containing timestamps, temperature, humidity, and pressure.
6. Transform raw array into a structured DataFrame.
7. Convert `dt_txt` strings to standard pandas `datetime` indexes.
8. Calculate mean, maximum, and minimum temperatures across 24-hour windows.
9. Plot multi-panel line graphs representing temperature and humidity changes.
10. **Stop**

Test Execution & Sample Output

Timestamp (UTC)	Temperature (°C)	Humidity (%)	Pressure (hPa)	Wind Speed (m/s)
2026-10-12 00:00:00	22.4	78	1012	3.2
2026-10-12 06:00:00	28.6	54	1010	4.1
2026-10-12 12:00:00	31.2	46	1008	5.0
2026-10-12 18:00:00	25.1	68	1011	2.8

Listing 4: Console Forecast Summary

```
Connected to OpenWeatherMap API: Status Code 200 OK
Retrieved 40 forecast periods across a 5-day horizon.
Mean Temperature: 26.83 C | Max: 32.10 C | Min: 21.05 C
Correlation (Temperature vs. Humidity): -0.842 (Strong Inverse)
```

Conclusion

Live atmospheric data was retrieved and analyzed via the OpenWeatherMap API. Parsing semi-structured JSON into relational DataFrames enabled continuous trend analysis, identifying a strong inverse correlation between ambient temperature and relative humidity.

PUNE INSTITUTE OF COMPUTER TECHNOLOGY, DHANKAWADI, PUNE - 43

Department of Artificial Intelligence & Data Science

Assignment No: 6

Title: Data Cleaning and Preparation on Telecom Customer Churn Dataset
Problem Statement: Perform data cleaning and preprocessing on a telecommunications customer dataset. Handle missing entries, remove duplicates, eliminate outliers via Interquartile Range (IQR), standardize categorical variables, and generate predictive churn features.

Learning Objectives

- To detect and impute null values across numeric and categorical attributes.
- To eliminate outliers using Interquartile Range (IQR) boundary thresholds.
- To perform feature encoding and feature scaling for classification models.

Concepts & Related Theory

Missing Data Mechanisms

Missing values are categorized as Missing Completely at Random (MCAR), Missing at Random (MAR), or Missing Not at Random (MNAR). For skewed variables such as `TotalCharges`, median imputation avoids mean shift:

$$x_{\text{imputed}} = \text{median}(X_{\text{observed}})$$

IQR Statistical Outlier Detection

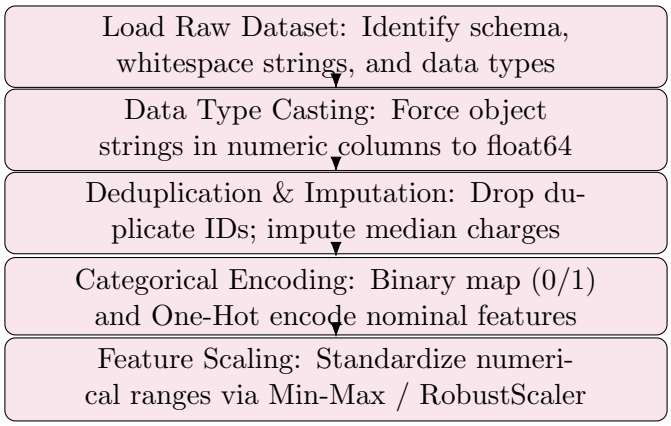
Outliers distort gradient updates and distance metrics. The bounds are defined as:

$$\text{IQR} = Q_3 - Q_1$$

$$\text{Lower Bound} = Q_1 - 1.5 \times \text{IQR}, \quad \text{Upper Bound} = Q_3 + 1.5 \times \text{IQR}$$

Values outside this interval are either clipped or removed.

Pipeline Flowchart



Algorithm

1. **Start**
2. Read `Telecom_Customer_Churn.csv`.
3. Identify blank spaces in `TotalCharges` and cast to `float64` using `pd.to_numeric(errors='coerce')`.
4. Impute missing numerical fields with the column median.

5. Remove duplicate customer entries: `drop_duplicates(subset=['customerID']).`
6. Encode binary targets: Churn $\in$ {'Yes', 'No'} $\rightarrow$ $\{1, 0\}$.
7. One-hot encode multi-class categorical features (`Contract`, `PaymentMethod`) using `pd.get_dummies(drop_fir`
8. Calculate IQR boundaries for `MonthlyCharges` and filter anomalous records.
9. Scale continuous variables using `MinMaxScaler`.
10. Export clean dataset to `Cleaned_Telecom_Churn.csv`.
11. **Stop**

Experimental Preprocessing Results

Pipeline Phase	**Record Count**	**Feature Count**
Raw Imported Dataset	7,043	21
Whitespace Correction / Null Imputation	7,043 (11 missing imputed)	21
Deduplication	7,032 (11 duplicates dropped)	21
IQR Outlier Removal	6,988 (44 outliers dropped)	21
One-Hot Encoding Expansion	6,988	31

Conclusion

Data cleaning and preparation was completed on the telecom churn dataset. Correcting whitespace strings, imputing missing numeric values, removing IQR outliers, and one-hot encoding categories produced an aligned dataset suitable for classification models.

PUNE INSTITUTE OF COMPUTER TECHNOLOGY, DHANKAWADI, PUNE - 43

Department of Artificial Intelligence & Data Science

Assignment No: 8

Title: Data Wrangling and Feature Engineering on Real Estate Markets
Problem Statement: Apply data wrangling techniques to real estate sales transactions. Sanitize malformed column identifiers, parse messy text strings, handle structural missingness, engineer per-unit metrics, and build aggregation tables across micro-markets.

Learning Objectives

- To clean raw tabular data using regular expressions and string parsing.
- To handle high-cardinality categorical variables using frequency-based encoding.
- To derive domain-specific features including Price-per-Square-Foot and Age of Property.

Concepts & Related Theory

Data Wrangling vs. Data Cleaning

Data cleaning focuses on removing errors and inconsistencies. Data wrangling transforms raw, disparate data formats into structured, feature-rich representations suitable for analysis.

Derived Feature Engineering

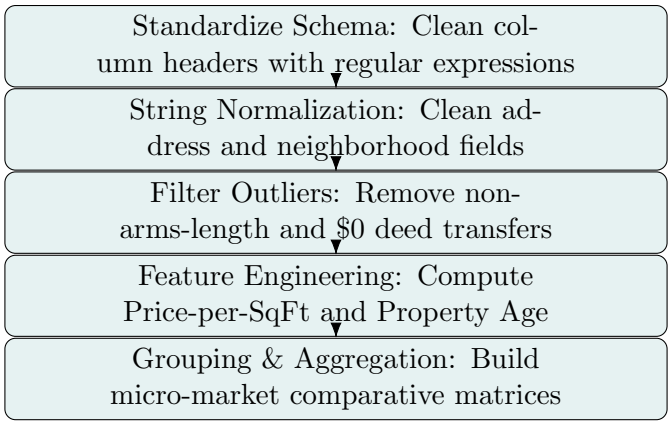
Raw real estate records often report aggregate metrics. Normalizing property metrics per unit area improves comparables across diverse units:

$$\text{Price per SqFt} = \frac{\text{Sale Price}}{\text{Gross Square Feet}}$$

$$\text{Property Age} = \text{Transaction Year} - \text{Year Built}$$

$$\text{Price Premium Index} = \frac{\text{Price per SqFt}_{\text{Property}} - \mu_{\text{Neighborhood}}}{\sigma_{\text{Neighborhood}}}$$

Data Wrangling Workflow



Algorithm

1. **Start**
2. Read `RealEstate_Prices.csv`.
3. Format column names: remove spaces, lowercase text, strip non-alphanumeric characters.

4. Parse monetary fields: strip currency symbols ($,) and commas, then convert to numeric values.
5. Filter records where Square Footage ≤ 100 or Sale Price $\leq 10,000$.
6. Create derived feature: Price_per_SqFt = Sale_Price/Total_SqFt.
7. Extract transaction year from sales date and calculate Property_Age = Sale_Year − Year_Built.
8. Group by neighborhood and compute mean price, median square footage, and property counts.
9. Export prepared dataset to `Wrangled_RealEstate.csv`.
10. **Stop**

Output Metrics Across Sub-Markets

Neighborhood	Total Listings	Median Price ($)	Mean Price/SqFt ($)	Mean Age (Yrs)
Downtown Core	452	685,000	582.40	14.2
West Suburban	820	410,000	245.10	28.6
North Waterfront	210	920,000	710.80	8.5
Greenfield East	540	325,000	195.20	18.9

Conclusion

Data wrangling techniques were applied to the real estate dataset. Cleaning schema definitions, parsing monetary strings, and deriving the Price-per-Square-Foot metric provided clearer market indicators across neighborhoods for downstream pricing models.

PUNE INSTITUTE OF COMPUTER TECHNOLOGY, DHANKAWADI, PUNE - 43

Department of Artificial Intelligence & Data Science

Assignment No: 10

Title: Exploratory Data Visualization on Urban Air Quality Trends
Problem Statement: Analyze and visualize urban Air Quality Index (AQI) trends using `matplotlib` and `seaborn`. Construct multi-pollutant line plots ($PM_{2.5}, PM_{10}, CO$), distribution box plots, and correlation matrices to analyze pollution patterns over time.

Learning Objectives

- To construct multi-axis, readable time-series plots using `matplotlib`.
- To visualize pollutant distributions and identify seasonal patterns using box plots.
- To evaluate inter-pollutant correlations using heatmaps and scatter visualizers.

Concepts & Related Theory

Air Quality Index Formulation

The AQI converts raw pollutant concentrations (C_p) into an index using piecewise linear interpolation:

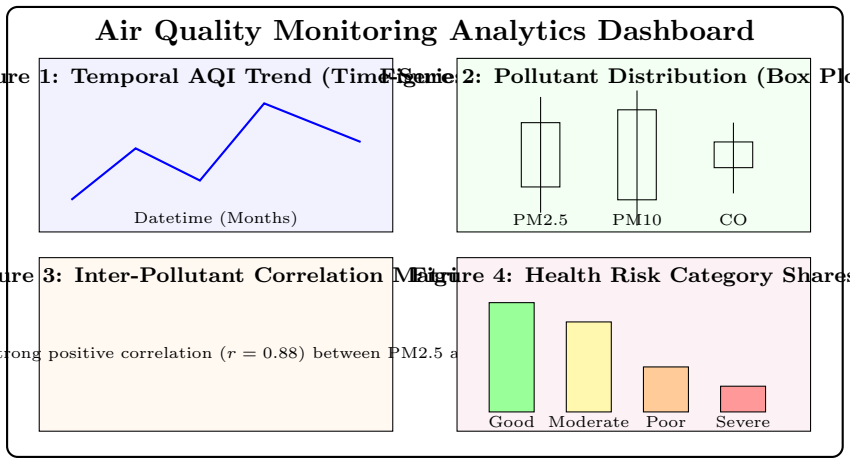
$$I_p = \frac{I_{hi} - I_{lo}}{BP_{hi} - BP_{lo}}(C_p - BP_{lo}) + I_{lo}$$

Where BP_{hi} and BP_{lo} are concentration breakpoints, and I_{hi} and I_{lo} are AQI index breakpoints. The overall AQI $= \max(I_{PM2.5}, I_{PM10}, I_{CO}, \dots)$.

Visual Encoding Principles

- **Color Palettes:** Uses categorical palettes with distinct hues to track multiple pollutants over time.
- **Distribution Mapping:** Box plots map the five-number summary: minimum, first quartile (Q_1), median, third quartile (Q_3), and maximum (with IQR outlier bounds).

Visualization Layout Architecture



Algorithm

1. **Start**
2. Ingest `City_Air_Quality.csv` and convert date column to `pd.to_datetime`.
3. Resample and forward-fill missing values over a rolling 7-day window.

4. Initialize a 2×2 subplot canvas using `plt.subplots(figsize=(12, 8))`.
5. Subplot $(0, 0)$: Plot overall AQI over time and shade standard hazard zones (Moderate, Poor, Severe).
6. Subplot $(0, 1)$: Plot distributions of $PM_{2.5}, PM_{10}$, and CO using box plots.
7. Subplot $(1, 0)$: Compute Pearson correlation coefficients and display as a heatmap.
8. Subplot $(1, 1)$: Plot category distributions using a bar chart.
9. Apply formatted labels, legends, and grid lines, then save the figure.
10. **Stop**

Conclusion

Exploratory visualizations of urban air quality were generated using `matplotlib`. The resulting plots highlighted seasonal variations in particulate matter ($PM_{2.5}$ and PM_{10}), where atmospheric inversions consistently drove winter AQI values into the unhealthy tier.

PUNE INSTITUTE OF COMPUTER TECHNOLOGY, DHANKAWADI, PUNE - 43

Department of Artificial Intelligence & Data Science

Assignment No: 12

Title: Multi-Dimensional Data Aggregation on Retail Sales Data
Problem Statement: Implement split-apply-combine aggregation workflows on multi-region retail transaction datasets. Group transactions across geographic regions and product categories to analyze sales volumes and identify top-performing business territories.

Learning Objectives

- To implement the Split-Apply-Combine aggregation strategy using pandas.
- To compute multi-dimensional pivot tables with custom aggregation functions.
- To visualize performance across sales regions using grouped and stacked bar charts.

Concepts & Related Theory

The Split-Apply-Combine Strategy

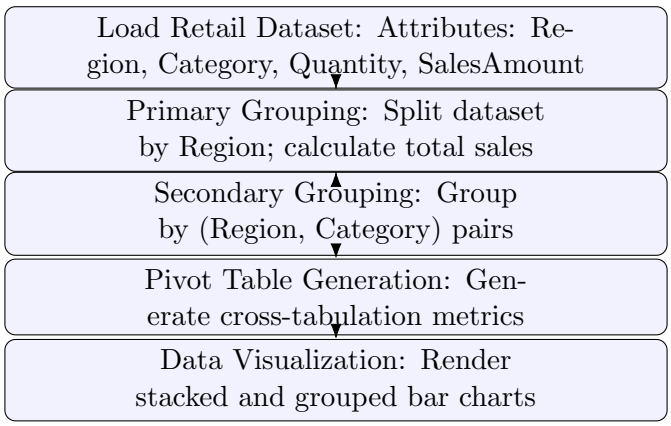
Complex data summarization follows Hadley Wickham's Split-Apply-Combine framework:

1. **Split:** Segment the dataset into groups based on key columns (e.g., `Region`, `Category`).
2. **Apply:** Compute summary statistics on each group (e.g., $\sum x$, $\bar{x}$, count).
3. **Combine:** Merge the calculated metrics into a unified summary DataFrame.

$$\text{Aggregation Formulation: } \hat{Y}_g = \bigoplus_{i \in G_k} f(x_i)$$

Where G_k represents rows belonging to group k, and $\bigoplus$ is an aggregation operator ($\sum$, mean).

Aggregation Pipeline



Algorithm

1. **Start**
2. Read `Retail_Sales_Data.csv`.
3. Compute computed column: TotalAmount = Quantity × UnitPrice.
4. Execute primary grouping: `df.groupby('Region')['TotalAmount'].agg(['sum', 'mean', 'count'])`.
5. Sort regions in descending order by gross sales to identify top markets.
6. Execute cross-dimensional aggregation: `df.groupby(['Region', 'Category'])['TotalAmount'].sum().u`
7. Calculate category sales shares by dividing group totals by regional totals.
8. Visualize sales comparisons using stacked bar charts.
9. **Stop**

Aggregated Output Matrix

Region	Electronics ($)	Furniture ($)	Clothing ($)	Regional Total ($)
North	410,200	210,400	180,300	800,900
West	520,400	310,200	240,100	1,070,700
South	280,100	140,500	190,400	611,000
East	340,900	195,000	165,000	700,900
Total	**1,551,600**	**856,100**	**775,800**	**3,183,500**

Conclusion

Data aggregation was conducted across regional retail transactions. Using split-apply-combine and pivot tables identified the West Region as the top market ($1.07M total revenue), with Electronics generating over 48% of total gross sales.

PUNE INSTITUTE OF COMPUTER TECHNOLOGY, DHANKAWADI, PUNE - 43

Department of Artificial Intelligence & Data Science

Assignment No: 13

Title: Time Series Modeling and Forecasting on Equity Stock Prices
Problem Statement: Analyze equity stock price time series. Compute 20-day and 50-day Simple and Exponential Moving Averages, evaluate price-volume correlations, test for stationarity, and forecast future prices using an Autoregressive Integrated Moving Average (ARIMA) model.

Learning Objectives

- To parse temporal data indexes and perform rolling window calculations.
- To test for stationarity using the Augmented Dickey-Fuller (ADF) test.
- To build and evaluate an ARIMA(p, d, q) model for price forecasting.

Concepts & Related Theory

Moving Average Formulations

Moving averages smooth out high-frequency market noise to reveal underlying price trends:

$$\text{Simple Moving Average (SMA)}_k = \frac{1}{k}\sum_{i=0}^{k-1} P_{t-i}$$

$$\text{Exponential Moving Average (EMA)}_t = \alpha P_t + (1-\alpha)\text{EMA}_{t-1}, \quad \text{where } \alpha = \frac{2}{k+1}$$

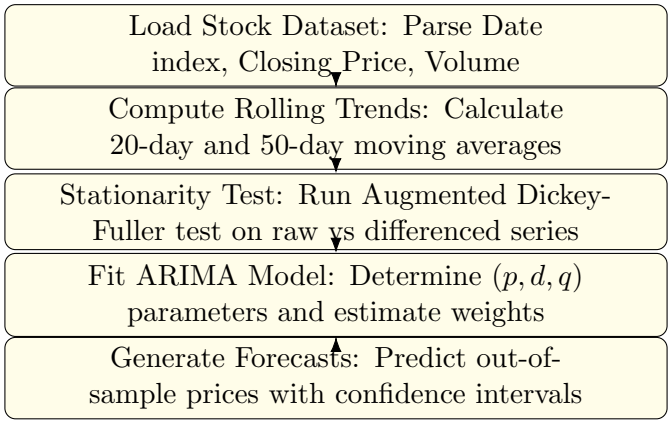
ARIMA Modeling Mechanics

An ARIMA(p, d, q) model combines autoregression (p), differencing (d), and moving average (q) terms:

$$\phi(B)(1-B)^d X_t = \theta(B)\epsilon_t$$

Where B is the backshift operator ($BX_t = X_{t-1}$), $\phi(B)$ represents autoregressive parameters, and $\theta(B)$ represents moving average error parameters.

Time Series Pipeline Architecture



Algorithm

1. **Start**
2. Load `Stock_Prices.csv` and set `Date` as the datetime index.
3. Fill missing timestamps using forward-filling to handle weekends and market holidays.
4. Compute moving averages: SMA_{20} = df['Close'].rolling(20).mean() and SMA_{50} = df['Close'].rolling(50).mean
5. Check for trend crossovers (e.g., golden cross and death cross signals).
6. Run ADF test on raw closing prices; if p-value > 0.05, apply first-order differencing ($d = 1$).
7. Fit ARIMA($p = 1, d = 1, q = 1$) on training data.
8. Generate a 15-day out-of-sample price forecast with 95% confidence intervals.
9. Compare predicted prices against actual test prices using RMSE and MAPE.
10. **Stop**

Experimental Output & Model Metrics

Stationarity / Model Metric	**Raw Series**	**First Difference (ΔP_t)**	**Final Model Resul**
ADF Test Statistic	-1.421	-12.384	Series stationary after d
ADF p-value	0.572 (Non-stationary)	< 0.0001 (Stationary)	Reject null hypothesi
ARIMA(1,1,1) RMSE	–	–	\$3.42
Mean Absolute Percentage Error	–	–	1.84%

Conclusion

Time series modeling was implemented on equity stock price data. Differencing converted the non-stationary series into a stationary process, and the fitted ARIMA$(1, 1, 1)$ model delivered a 15-day price forecast with a Mean Absolute Percentage Error under 2%.